Architecture Documentation

This document describes the system architecture of The Autonomous AI Startup, including component interactions, data flow, and design decisions.

System Overview

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        AUTONOMOUS AI STARTUP                             │
│                      Phase 3 Complete - 112+ Agents                      │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   ┌──────────────────┐    ┌──────────────────┐    ┌──────────────────┐  │
│   │  Next.js 16      │    │   SQLite DB      │    │  Agency-Agents   │  │
│   │  App Router      │────│  (WAL mode)      │────│  Repository      │  │
│   └──────────────────┘    └──────────────────┘    └──────────────────┘  │
│          │                      │                        │              │
│          │                      │                        │              │
│   ┌──────────────────────────────────────────────────────────────────┐  │
│   │                        CORE LIBRARIES                            │  │
│   ├──────────────────────────────────────────────────────────────────┤  │
│   │ ├── task-planner.ts     (Enhanced task routing & collab)         │  │
│   │ ├── agent-hiring.ts     (Dynamic agent creation)                 │  │
│   │ ├── agent-importer.ts   (Import from agency-agents)              │  │
│   │ ├── agent-sync.ts       (Workspace synchronization)              │  │
│   │ ├── websocket.ts        (Gateway connection)                     │  │
│   │ └── device-identity.ts  (Ed25519 identity)                       │  │
│   └──────────────────────────────────────────────────────────────────┘  │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐  │
│   │                      AGENT LAYER (112+ Agents)                   │  │
│   ├──────────────────────────────────────────────────────────────────┤  │
│   │ 🎮 Game Dev (19) │ 💻 Engineering (18) │ 📣 Marketing (18)       │  │
│   │ 🧪 Testing (8)   │ 🎨 Design (8)       │ 💰 Paid Media (7)       │  │
│   │ ⚙️ Operations (7)│ 📋 Project Mgmt (6) │ 💬 Support (6)          │  │
│   │ 🌐 Spatial (6)   │ 📦 Product (4)      │ ♟️ Strategy (3)         │  │
│   │ 👔 Executive (1) │ 💼 Sales (1)        │                         │  │
│   └──────────────────────────────────────────────────────────────────┘  │
│                                  │                                       │
│                                  ▼                                       │
│   ┌──────────────────────────────────────────────────────────────────┐  │
│   │                        OpenClaw Gateway                          │  │
│   │                   ws://127.0.0.1:18789 (v3)                      │  │
│   └──────────────────────────────────────────────────────────────────┘  │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Core Components

1. Task Planner (Enhanced Orchestrator)

Location: `src/lib/task-planner.ts`

The Task Planner is the brain of the system, now enhanced with Phase 2 features:

Core Responsibilities

- **Task Classification:** Analyzes tasks to determine type and complexity
- **Agent Matching:** Scores agents based on division, specialization, and workload
- **Multi-Agent Collaboration:** Coordinates tasks that require multiple agents
- **Task Dependencies:** Manages complex workflows with task dependencies
- **Workload Balancing:** Redistributes tasks to prevent agent overload

Task Routing Flow

```
User Request
    │
    ▼
Task Planner
    │
    ├─ Classify Task (type, complexity)
    │
    ├─ Check Collaboration Need
    │       │
    │       └─ If multi-agent → Create Collaboration
    │
    ├─ Score Available Agents
    │       │
    │       ├─ Division match
    │       ├─ Specialization bonus
    │       ├─ Workload factor
    │       └─ Complexity assessment
    │
    ├─ Best match found?
    │       │
    │       ├─ Yes → Assign to Agent
    │       └─ No → Trigger Autonomous Hiring
    │
    └─ Create Task Dependencies (if complex)
```

Complexity Assessment

```
// Complexity factors
- Multi-step keywords: 'comprehensive', 'full-stack', 'end-to-end'
- Integration keywords: 'integrate', 'connect', 'sync'
- Advanced keywords: 'optimize', 'architect', 'design system'
- Scale keywords: 'enterprise', 'production', 'at scale'
```

2. Agent Import System (Phase 2)

Location: `src/lib/agent-importer.ts`

Imports specialized agents from the agency-agents repository:

Import Flow

```
agency-agents Repository
          │
          ▼
    AgentImporter
          │
    ├── fetchDivisionAgents() → List available .md files
    ├── fetchAgentMarkdown()  → Download agent content
    ├── parseAgentMarkdown()  → Extract metadata & capabilities
    ├── saveAgentToWorkspace() → Create soul.md file
    └── importAgentToDb()     → Insert/update database record
```

Division Mapping

Source Folder	System Division
engineering	engineering
design	design
marketing	marketing
paid-media	paid-media
product	product
project-management	project-management
testing	testing
support	support
spatial-computing	spatial-computing
game-development	game-development
strategy	strategy

3. Agent Hiring Framework (Enhanced)

Location: `src/lib/agent-hiring.ts`

Dynamic agent creation with Phase 2 enhancements:

Features

- **Capability Detection:** Enhanced scoring for task requirements
- **Division Detection:** Automatically determines best division
- **Template Matching:** Uses expanded AGENT_TEMPLATES
- **Autonomous Hiring:** Can auto-create high-priority agents
- **Integration:** Works with AgentImporter for workspace setup

Hiring Flow

```
Task Analysis
     │
     ├── detectCapabilities() → Required skills
     ├── detectDivision()     → Best-fit division
     ├── findMatchingAgents() → Existing matches
     │
     └── evaluateHiringNeed()
              │
              ├── High confidence → Auto-create agent
              ├── Medium → Create hiring request
              └── Low → Manual review required
```

4. Agent SOUL System

Location: `workspace/agents/*/soul.md`

Each agent has a SOUL (System Of Unique Learning) file:

```yaml
---
name: "Agent Name"
description: "Agent description"
emoji: "🤖"
color: "#3b82f6"
division: "engineering"
specialization: "Backend Architecture"
source: "agency-agents"
source_url: "https://..."
vibe: "professional, skilled, collaborative"
---

# Agent Name

## Your Identity
...

## Core Capabilities
...

## Critical Rules
...
```

5. Database Layer (Extended Schema)

Location: `src/lib/db.ts`

SQLite database with Phase 2 schema extensions:

Table	Purpose	Phase 2 Additions
`agents`	Agent profiles	`specialization`, `source`, `technical_skills`, `personality_traits`
`tasks`	Task queue	`dependencies`, `estimated_hours`, `tags`
`task_dependencies`	Task workflow	**New table**
`agent_collaborations`	Multi-agent work	**New table**
`import_history`	Import tracking	**New table**
`hiring_requests`	Creation requests	`suggested_division`, `priority`, `justification`

6. State Management

Location: `src/store/index.ts`

Zustand store with expanded state:

```
interface AppState {
  agents: Agent[];           // 112+ agents
  tasks: Task[];
  events: AppEvent[];
  gatewayConnection: GatewayConnection | null;

  // UI state
  activePanel: string;
  sidebarOpen: boolean;
  isLoading: boolean;

  // Selectors
  selectAgentById: (id: string) => Agent | undefined;
  selectAgentsByDivision: (division: AgentDivision) => Agent[];
  // ...
}
```

7. OpenClaw Integration

Location: `src/lib/websocket.ts`, `src/lib/device-identity.ts`

- **WebSocket Client:** Protocol v3 connection to OpenClaw Gateway
- **Device Identity:** Ed25519 key pair for secure authentication
- **Agent Sync:** Bidirectional sync with OpenClaw configuration

8. Visual Task Pipeline Dashboard

Location: `src/components/panels/PipelinePanel.tsx`

Real-time Kanban-style visualization of task flow through the system:

Pipeline Stages

```
┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│  🎯 Planner  │ → │  📋 Tasks    │ → │  🤖 Agents   │ → │  ✨ Results  │
│              │   │              │   │              │   │              │
│ Planning &   │   │ Queued &     │   │ Active       │   │ Review &     │
│ Analysis     │   │ Backlog      │   │ Execution    │   │ Complete     │
│              │   │              │   │              │   │              │
│ inbox        │   │ backlog      │   │ in_progress  │   │ review       │
│              │   │ todo         │   │              │   │ done         │
│              │   │ blocked      │   │              │   │              │
└──────────────┘   └──────────────┘   └──────────────┘   └──────────────┘
```

Features

- **Real-time Updates:** Live task movement across stages
- **Metrics Dashboard:** Total tasks, in-progress, completed, blocked, completion rate
- **Filtering:** By status and project/tag
- **Task Details:** Click any task for detailed sidebar view
- **Agent Assignment:** Visual indication of which agent is working on each task
- **Dependencies:** Visual indicators for task dependencies

9. IDEA → STARTUP Generator

Location: `src/lib/startup-generator.ts`, `src/app/api/startup/generate/route.ts`

One-click startup generation from an idea:

Pipeline Phases

```
Phase 1: Product Definition
  └── product-manager, market-researcher

Phase 2: Design
  └── ux-designer, product-manager

Phase 3: Build
  └── software-architect, backend-developer, frontend-developer

Phase 4: Marketing
  └── copywriter, seo-strategist, growth-hacker
```

Generation Flow

```
User Idea Input
      │
      ▼
StartupGenerator
      │
      ├── Generate project ID & structure
      ├── Create project brief (brief.md)
      ├── Parse startup_pipeline.yaml template
      ├── Create tasks with dependencies
      ├── Assign appropriate agents
      └── Add tasks to queue
            │
            ▼
     Pipeline Dashboard (visualize progress)
```

Data Flow

Multi-Agent Collaboration Flow

```
Complex Task Created
        │
        ▼
Task Planner Analyzes
        │
        ├── Requires multiple agents?
        │         │
        │         └── Yes → createCollaboration()
        │                        │
        │                        ├── Identify required divisions
        │                        ├── Select lead agent
        │                        ├── Select supporting agents
        │                        └── Create collaboration record
        │
        └── Decompose into subtasks
                  │
                  ├── Create task dependencies
                  ├── Assign subtasks to agents
                  └── Set up dependency tracking
```

Task Dependency Management

```
┌──────────────────────────────────────────────────────────┐
│                    Task Dependency Flow                  │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Task A (Research) ──finish_to_start──→ Task B (Design)│
│                                                 │        │
│                                        finish_to_start   │
│                                                 │        │
│                                                 ▼        │
│                                       Task C (Implement) │
│                                                 │        │
│                                        finish_to_start   │
│                                                 │        │
│                                                 ▼        │
│                                        Task D (Test)     │
│                                                          │
└──────────────────────────────────────────────────────────┘

Dependency Types:
- finish_to_start: B starts after A finishes (most common)
- start_to_start: B starts when A starts
- finish_to_finish: B finishes when A finishes
- start_to_finish: B finishes when A starts
```

Agent Import Flow

```
Import Trigger (script or API)
        │
        ▼
AgentImporter.importDivision()
        │
        ├── Fetch agent list from GitHub
        │
        ├── For each agent .md file:
        │       │
        │       ├── Download content
        │       ├── Parse markdown to ParsedAgent
        │       │       ├── Extract name, description
        │       │       ├── Detect emoji, color
        │       │       ├── Extract capabilities
        │       │       ├── Extract technical skills
        │       │       └── Extract personality traits
        │       │
        │       ├── Create workspace directory
        │       ├── Generate soul.md file
        │       └── Insert/update database
        │
        └── Record import history
```

Design Decisions

Why SQLite?

1. **Zero external dependencies** - No database server to manage
2. **WAL mode** - High concurrency support
3. **Portability** - Single file, easy backup

4. **Performance** - Excellent for read-heavy workloads

Why 14 Divisions?

Divisions align with agency-agents repository structure and common organizational needs:
- Engineering, Design, Marketing (core functions)
- Game Development, Spatial Computing (specialized tech)
- Testing, Support, Operations (quality & support)
- Strategy, Executive, Sales (leadership & revenue)

Why Multi-Agent Collaboration?

Complex tasks often require diverse expertise:
- A marketing campaign needs content, design, and paid media
- A product launch needs engineering, QA, and marketing
- System design needs architecture, security, and DevOps

Why Task Dependencies?

Real workflows have ordering requirements:
- Can’t test before implementing
- Can’t implement before designing
- Can’t deploy before testing

Phase 3: OpenClaw Integration

8. OpenClaw Client (Enhanced)

Location: `src/lib/openclaw-client.ts`

Full-featured WebSocket client for OpenClaw gateway:

```
┌──────────────────────────────────────────────────────────────────┐
│                      OpenClaw Client                             │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────────┐     ┌──────────────────┐                 │
│   │ Connection Mgmt  │     │ Authentication   │                 │
│   │ - Auto-reconnect│     │ - Ed25519 keys   │                 │
│   │ - Exponential    │     │ - Challenge/     │                 │
│   │   backoff        │     │   response       │                 │
│   │ - Ping/pong      │     │ - Token auth     │                 │
│   └──────────────────┘     └──────────────────┘                 │
│                                                                  │
│   ┌──────────────────┐     ┌──────────────────┐                 │
│   │ Message Queue    │     │ Event Emitter    │                 │
│   │ - Offline queue  │     │ - connected      │                 │
│   │ - Retry logic    │     │ - message        │                 │
│   │ - Max queue      │     │ - error          │                 │
│   │   management     │     │ - reconnecting   │                 │
│   └──────────────────┘     └──────────────────┘                 │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Connection States

```
type ConnectionState =
  | 'disconnected'    // Initial or after disconnect
  | 'connecting'      // Establishing WebSocket
  | 'authenticating'  // Sending auth payload
  | 'connected'       // Ready for messages
  | 'reconnecting'    // Auto-reconnect in progress
  | 'error';          // Connection failed
```

9. Message Router

Location: `src/lib/message-router.ts`

Routes messages between OpenClaw channels and agents:

```
Incoming Channel Message
         │
         ▼
┌──────────────────────┐
│   Message Router     │
├──────────────────────┤
│ 1. Parse message     │
│ 2. Identify channel  │
│ 3. Apply filters     │
│ 4. Match agents      │
│ 5. Route to agents   │
│ 6. Format response   │
│ 7. Send to channel   │
└──────────────────────┘
         │
         ▼
Agent Response → Channel
```

Routing Decision

```
interface RoutingDecision {
  shouldProcess: boolean;
  targetAgents: string[];
  reason: string;
  priority: 'high' | 'medium' | 'low';
}
```

10. Channel Configuration

Location: `src/lib/openclaw-config.ts`

Manages multi-channel connectivity:

Platform	Config Options
Slack	workspace, channel
Discord	guild, channel
Telegram	chatId
WhatsApp	number
Teams	webhookUrl
Email	address
Matrix	metadata
Webchat	apiKey

Agent-Channel Mapping

```
interface AgentChannelMapping {
  agentSlug: string;
  role: 'responder' | 'listener' | 'broadcaster';
  filter?: MessageFilter;
}
```

11. Analytics System

Location: `src/lib/analytics.ts`

Comprehensive system monitoring:

```
┌──────────────────────────────────────────────────────────────────┐
│                     Analytics Dashboard                          │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                    System Health                         │   │
│   │  • Database size & table count                           │   │
│   │  • Active agents by division                             │   │
│   │  • Connection status                                     │   │
│   │  • Uptime tracking                                       │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                   Agent Metrics                          │   │
│   │  • Tasks completed/failed                                │   │
│   │  • Success rate                                          │   │
│   │  • Average response time                                 │   │
│   │  • Task duration                                         │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                    Task Analytics                        │   │
│   │  • Status distribution                                   │   │
│   │  • Priority breakdown                                    │   │
│   │  • Bottleneck detection                                  │   │
│   │  • Throughput (daily/weekly/monthly)                     │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

12. Caching Layer

Location: `src/lib/cache.ts`

In-memory caching with TTL:

```
interface CacheConfig {
  maxSize: number;        // Maximum cached items
  defaultTTL: number;     // Default TTL in seconds
  cleanupInterval: number; // Cleanup frequency
}
```

Cache Usage

- Analytics queries (30s TTL)
- Agent metrics (60s TTL)
- System health (10s TTL)
- Configuration data (300s TTL)

13. Performance Monitor

Location: `src/lib/performance-monitor.ts`

Tracks system performance:

```typescript
interface PerformanceMetrics {
  responseTime: {
    avg: number;
    p50: number;
    p95: number;
    p99: number;
  };
  throughput: {
    requestsPerSecond: number;
    tasksPerHour: number;
  };
  cache: {
    hitRate: number;
    size: number;
    maxSize: number;
  };
  errors: {
    rate: number;
    count: number;
    byType: Record<string, number>;
  };
}
```

14. Database Optimizer

Location: `src/lib/db-optimizer.ts`

Database maintenance utilities:

- **VACUUM** - Reclaim disk space
- **ANALYZE** - Update query planner statistics
- **Index management** - Create/verify indexes
- **WAL checkpoint** - Manage write-ahead log
- **Integrity checks** - Detect corruption

Security Considerations

- Device identity keys stored locally (not in git)
- API key authentication for headless access
- Session-based auth with scrypt hashing
- CSRF protection enabled
- Input validation with Zod
- Agent source URL tracking for audit
- Ed25519 cryptographic signatures for OpenClaw
- WSS (WebSocket Secure) for production
- Message queue overflow protection

Performance Considerations

- SQLite WAL mode for concurrent reads
- Workload balancing prevents agent overload
- Lazy loading of agent data
- Efficient task priority queue

- Import rate limiting for GitHub API
- In-memory caching with TTL eviction
- Database query optimization
- Connection pooling for WebSocket
- Automatic garbage collection for old data

Operational Layer

The Operational Layer provides a Python-based runtime for autonomous task execution, enabling Mission Control to orchestrate agents via the OpenClaw Gateway.

Architecture Overview

```
╔═══════════════════════════════════════════════════════════════════════╗
║                          OPERATIONAL LAYER                            ╕
╞═══════════════════════════════════════════════════════════════════════╡
│                                                                       │
│   run_mission_control.py                                              │
│   ┌─────────────────────────────────────────────────────────────┐     │
│   │                       Mission Control                       │     │
│   │   • Configuration loading (YAML/env vars)                   │     │
│   │   • Signal handling (graceful shutdown)                     │     │
│   │   • Component initialization                                │     │
│   │   • Main execution loop                                     │     │
│   └─────────────────────────────────────────────────────────────┘     │
│                                │                                      │
│          ┌─────────────────────┴─────────────────────┐                │
│          ▼                     ▼                     ▼                │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────┐         │
│   │   OpenClaw   │     │    Task      │     │   Project    │         │
│   │   Connector  │     │    Runner    │     │   Memory     │         │
│   └──────────────┘     └──────────────┘     └──────────────┘         │
│          │                     │                     │                │
│          └─────────────────────┬─────────────────────┘                │
│                                ▼                                      │
│                       ┌──────────────┐                                │
│                       │    Logger    │                                │
│                       │ (Structured) │                                │
│                       └──────────────┘                                │
│                                                                       │
╘═══════════════════════════════════════════════════════════════════════╛
                                │
                                ▼
╔═══════════════════════════════════════════════════════════════════════╗
║                   OpenClaw Gateway (External)                         ╕
│                    ws://127.0.0.1:18789                               │
╘═══════════════════════════════════════════════════════════════════════╛
```

15. OpenClaw Connector (Python)

Location: `connectors/openclaw_client.py`

Python client for communicating with the OpenClaw Gateway:

```python
class OpenClawClient:
    """OpenClaw Gateway client for task execution."""

    def __init__(self, api_endpoint, api_token, timeout, retry_attempts):
        # Configuration
        self.api_endpoint = api_endpoint
        self.api_token = api_token

    def health_check(self) -> bool:
        """Check if OpenClaw Gateway is reachable."""

    def send_task(self, task: dict) -> dict:
        """Send a task to OpenClaw for agent execution."""

    def get_task_status(self, task_id: str) -> dict:
        """Poll for task completion status."""
```

Features

- HTTP/REST-based communication
- Automatic retry with exponential backoff
- Configurable timeout
- Health check endpoint
- Task submission and status polling

16. Task Runner (Execution Loop)

Location: `orchestration/task_runner.py`

Core execution loop for processing tasks:

```
┌──────────────────────────────────────────────────────────────┐
│                      Task Runner Loop                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   start()                                                    │
│       │                                                      │
│       ▼                                                      │
│   ┌──────────────────────────────────────────────────────┐   │
│   │  while running:                                      │   │
│   │      │                                               │   │
│   │      ├── fetch_pending_tasks()                       │   │
│   │      │       │                                       │   │
│   │      │       └── Query tasks with status='todo'      │   │
│   │      │                                               │   │
│   │      ├── for task in tasks:                          │   │
│   │      │       │                                       │   │
│   │      │       ├── update_status('in_progress')        │   │
│   │      │       │                                       │   │
│   │      │       ├── execute_via_openclaw(task)          │   │
│   │      │       │       │                               │   │
│   │      │       │       └── Submit to OpenClaw Gateway  │   │
│   │      │       │                                       │   │
│   │      │       ├── wait_for_completion()               │   │
│   │      │       │                                       │   │
│   │      │       └── update_status('done'|'failed')      │   │
│   │      │                                               │   │
│   │      └── sleep(poll_interval)                        │   │
│   │                                                      │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Task States

State	Description
`todo`	Task queued, waiting for execution
`in_progress`	Task currently being executed
`done`	Task completed successfully
`failed`	Task execution failed
`cancelled`	Task was cancelled

17. Project Memory Bridge

Location: `orchestration/project_memory.py`

Persistent context and state tracking:

```python
class ProjectMemory:
    """Maintains project context across task executions."""

    def __init__(self, storage_path: str):
        self.storage_path = storage_path
        self.context = {}

    def load(self) -> dict:
        """Load persisted project memory."""

    def save(self):
        """Persist current memory state."""

    def update_context(self, key: str, value: Any):
        """Update a specific context key."""

    def get_context(self, key: str) -> Any:
        """Retrieve a context value."""
```

18. Logging System

Location: `orchestration/logger.py`

Structured logging with configurable output:

```python
def get_logger(log_level: str = 'INFO', console_output: bool = True) -> Logger:
    """Create a configured logger instance."""
```

Log Format

```
2026-03-11 12:00:00,000 [] INFO [] mission_control [] Initializing Mission Control...
2026-03-11 12:00:01,000 [] INFO [] openclaw_client [] Connected to OpenClaw Gateway
2026-03-11 12:00:02,000 [] INFO [] task_runner [] Starting task execution loop
```

How Mission Control Orchestrates Agents

```
╔═══════════════════════════════════════════════════════════════════╗
║═══════════════════════════════════════════════════════════════════╕
│                     Mission Control Flow                          │
╞═══════════════════════════════════════════════════════════════════╡
│                                                                   │
│  1. User/System creates task                                      │
│         │                                                         │
│         ▼                                                         │
│  2. Task stored in SQLite (status: 'todo')                        │
│         │                                                         │
│         ▼                                                         │
│  3. Task Runner polls for pending tasks                           │
│         │                                                         │
│         ▼                                                         │
│  4. Task assigned to agent (via Task Planner)                     │
│         │                                                         │
│         ▼                                                         │
│  5. OpenClaw Connector sends task to Gateway                      │
│         │                                                         │
│         ▼                                                         │
│  6. OpenClaw routes to appropriate agent channel                  │
│         │                                                         │
│         ▼                                                         │
│  7. Agent processes task, generates response                      │
│         │                                                         │
│         ▼                                                         │
│  8. Response returned via OpenClaw                                │
│         │                                                         │
│         ▼                                                         │
│  9. Task Runner updates task status                               │
│         │                                                         │
│         ▼                                                         │
│  10. Project Memory updated with context                          │
│                                                                   │
╘═══════════════════════════════════════════════════════════════════╛
```

Environment Variables

Variable	Description	Default
`OPENCLAW_GATEWAY_HOST`	Gateway hostname	`localhost`
`OPENCLAW_GATEWAY_PORT`	Gateway port	`8080`
`OPENCLAW_GATEWAY_TOKEN`	Authentication token	(none)
`MC_CONFIG`	Config file path	`config.yaml`
`LOG_LEVEL`	Logging level	`INFO`

Operational Layer Improvements (v2)

The following enhancements extend the operational layer with advanced features for scalability, reliability, and observability.

19. Task Dependency Graph

Location: `orchestration/task_dependencies.py`

Enables tasks to specify dependencies on other tasks:

```
class DependencyManager:
    """Manages task dependencies and execution order."""

    def can_execute(self, task_id: str) -> bool:
        """Check if all dependencies are satisfied."""

    def get_ready_tasks(self) -> List[str]:
        """Get all tasks ready for execution."""

    def check_dependencies(self, task_id: str) -> DependencyInfo:
        """Get detailed dependency status."""

    def build_dependency_graph(self) -> Dict[str, List[str]]:
        """Build complete dependency graph."""
```

Task JSON with Dependencies:

```
{
  "task_id": "task_frontend",
  "depends_on": ["task_design", "task_api"],
  "description": "Build frontend components"
}
```

Dependency States:

State	Description
`satisfied`	All dependencies completed successfully
`pending`	Some dependencies not yet complete
`blocked`	A dependency failed
`missing`	A dependency task not found

20. Execution Run Sessions

Location: `orchestration/run_sessions.py`

Isolated run sessions for tracking each Mission Control execution:

```
runs/
└── run_20260311_143052_a1b2c3d4/
    ├── metadata.json    # Run status, config, timestamps
    ├── tasks.json       # All tasks in this run
    ├── logs.json        # Structured event logs
    ├── metrics.json     # Performance metrics
    └── outputs/
        └── {task_id}/
            └── result.json
```

```python
class RunSession:
    """Manages a single execution run session."""

    def add_task(self, task_id: str, task_data: dict):
        """Track a task in this run."""

    def log_event(self, event_type: str, data: dict):
        """Log an event to the run."""

    def save_output(self, task_id: str, output: Any):
        """Save agent output for a task."""

    def record_metric(self, name: str, value: Any):
        """Record a metric value."""
```

21. Agent Capability Index

Location: `orchestration/capability_index.py`

Matches tasks to agents based on capabilities parsed from soul.md files:

```python
class CapabilityIndex:
    """Index of agent capabilities for task routing."""

    def load_agents(self):
        """Load capabilities from workspace/agents."""

    def match_task_to_agents(self, task: dict, limit: int = 5) ->
List[CapabilityMatch]:
        """Find best agents for a task based on capabilities."""

    def get_agents_by_capability(self, capability: str) -> List[str]:
        """Get agents with a specific capability."""
```

Capability Match Scoring:
- Division match: +0.4
- Keyword overlap: up to +0.3
- Direct capability match: +0.15 each
- Technical skill match: +0.1 each
- Specialization bonus: +0.2

22. Scheduler Layer

Location: `orchestration/scheduler.py`

Dependency-aware scheduler with concurrency and rate control:

```
┌──────────────────────────────────────────────────────────────────────┐
│                         Scheduler Flow                               │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   1. Poll backlog tasks                                              │
│         │                                                            │
│         ▼                                                            │
│   2. Check dependencies for each task                                │
│         │                                                            │
│         ├── Satisfied → Add to ready queue                           │
│         └── Pending/Blocked → Keep in backlog                        │
│                                                                      │
│   3. Sort ready tasks by priority and creation time                  │
│         │                                                            │
│         ▼                                                            │
│   4. Check concurrency limit                                         │
│         │                                                            │
│         ▼                                                            │
│   5. Check rate limit                                                │
│         │                                                            │
│         ▼                                                            │
│   6. Dispatch to Task Runner                                         │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

```python
class Scheduler:
    """Task scheduler with dependency resolution."""

    PRIORITY_CRITICAL = 0
    PRIORITY_HIGH = 1
    PRIORITY_NORMAL = 2
    PRIORITY_LOW = 3

    def start(self, blocking: bool = False):
        """Start the scheduler."""

    def stop(self):
        """Stop the scheduler."""

    def get_status(self) -> dict:
        """Get scheduler status."""
```

Configuration:

```yaml
scheduler:
  enabled: true
  poll_interval: 2.0      # Seconds between polls
  rate_limit: 0           # Tasks per second (0 = unlimited)

task_processing:
  max_concurrent: 5       # Parallel task limit
```

23. Enhanced Observability

Locations:

- `orchestration/metrics.py` - Metrics collection
- `api/metrics_api.py` - HTTP API

Tracked Metrics:

Metric	Type	Description
`tasks_created`	Counter	Total tasks created
`tasks_completed`	Counter	Successfully completed tasks
`tasks_failed`	Counter	Failed tasks
`tasks_dispatched`	Counter	Tasks sent to agents
`agent_execution_time`	Timer	Execution duration per agent
`scheduler_errors`	Counter	Scheduling errors
`queue_depth`	Gauge	Current backlog size

```python
from orchestration.metrics import get_metrics_collector

metrics = get_metrics_collector()

# Increment counters
metrics.increment('tasks_completed')

# Record timers
with metrics.time('agent_execution_time'):
    # execute task
    pass

# Get all metrics
data = metrics.get_all_metrics()
```

Metrics API Endpoints:

```
GET /metrics          # All current metrics
GET /metrics/summary  # Metric summaries with stats
GET /metrics/history  # Recent metric history
GET /health           # Health check
GET /status           # System status (scheduler, run)
GET /runs             # List recent runs
```

Running with Metrics API:

```
python run_mission_control.py --metrics-port 9090
```

Updated Architecture Diagram

```
╔═══════════════════════════════════════════════════════════════════════════╗
║                         OPERATIONAL LAYER v2                              ║
╚═══════════════════════════════════════════════════════════════════════════╝
│                                                                           │
│   run_mission_control.py                                                  │
│   ┌─────────────────────────────────────────────────────────────────────┐
│   │                        Mission Control                              │ │
│   │  • Run Session management                                           │ │
│   │  • Metrics collection                                               │ │
│   │  • Capability index loading                                         │ │
│   └─────────────────────────────────────────────────────────────────────┘
│                                │
│   ┌─────────────────────────────────────────────────────────────────────┐
│   │                          Scheduler                                  │ │
│   │  • Dependency resolution                                            │ │
│   │  • Priority ordering                                                │ │
│   │  • Rate limiting                                                    │ │
│   └─────────────────────────────────────────────────────────────────────┘
│                                │
│           ┌────────────────────┴────────────────────┐
│           ▼                    ▼                    ▼
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│   │   OpenClaw   │   │    Task      │   │  Capability  │
│   │   Connector  │   │    Runner    │   │    Index     │
│   └──────────────┘   └──────────────┘   └──────────────┘
│           │                    │                    │
│           └────────────────────┬────────────────────┘
│                                │
│   ┌─────────────────────────────────────────────────────────────────────┐
│   │                     Run Session + Metrics                           │ │
│   │  runs/{run_id}/ │ logs.json │ metrics.json │ outputs/               │ │
│   └─────────────────────────────────────────────────────────────────────┘
│                                │
│   ┌─────────────────────────────────────────────────────────────────────┐
│   │                       Metrics API (:9090)                           │ │
│   │  /metrics │ /metrics/summary │ /status │ /runs                      │ │
│   └─────────────────────────────────────────────────────────────────────┘
│                                                                           │
└───────────────────────────────────────────────────────────────────────────┘
```

Agent Knowledge & Reputation System

The knowledge and reputation system enables agents to learn from past projects and improves task routing based on performance history.

Architecture Overview

```
╔═══════════════════════════════════════════════════════════════════════╗
║                    KNOWLEDGE & REPUTATION SYSTEM                      ║
╚═══════════════════════════════════════════════════════════════════════╝

    ╔═══════════════════════════════════════════════════════════════╗
    ║                    Task Runner (Enhanced)                     ║
    ║                                                               ║
    ║  BEFORE EXECUTION:                                            ║
    ║  ┌─────────────────────────────────────────────────────────┐  ║
    ║  │ 1. Get task description                                 │  ║
    ║  │ 2. Call get_knowledge_context(task_description)         │  ║
    ║  │ 3. Inject retrieved knowledge into agent context        │  ║
    ║  └─────────────────────────────────────────────────────────┘  ║
    ║                                                               ║
    ║  AFTER EXECUTION:                                             ║
    ║  ┌─────────────────────────────────────────────────────────┐  ║
    ║  │ 4. Call store_agent_knowledge(agent, output, project)   │  ║
    ║  │ 5. Call update_agent_reputation(agent, success, time)   │  ║
    ║  └─────────────────────────────────────────────────────────┘  ║
    ╚═══════════════════════════════════════════════════════════════╝
                                  │
                      ┌───────────┴───────────┐
                      ▼                       ▼                       ▼
    ┌─────────────┐   ┌─────────────┐   ┌─────────────┐
    │ Knowledge   │   │ Embedding   │   │ Reputation  │
    │  Capture    │   │  System     │   │  Manager    │
    └─────────────┘   └─────────────┘   └─────────────┘
                      │                       │
                      └───────────┬───────────┘
                                  │
    ╔═══════════════════════════════════════════════════════════════╗
    ║                    SQLite Database                            ║
    ║  ┌──────────────────────┐  ┌──────────────────────┐           ║
    ║  │ knowledge_entries    │  │ agent_reputation     │           ║
    ║  │ - id                 │  │ - agent_name (PK)    │           ║
    ║  │ - agent_name         │  │ - tasks_completed    │           ║
    ║  │ - knowledge_type     │  │ - tasks_failed       │           ║
    ║  │ - domain             │  │ - total_exec_time    │           ║
    ║  │ - content            │  │ - avg_exec_time      │           ║
    ║  │ - embedding          │  │ - success_rate       │           ║
    ║  │ - project_id         │  │ - last_updated       │           ║
    ║  │ - created_at         │  │                      │           ║
    ║  └──────────────────────┘  └──────────────────────┘           ║
    ╚═══════════════════════════════════════════════════════════════╝
                                  │
    ╔═══════════════════════════════════════════════════════════════╗
    ║                    Markdown Knowledge Files                   ║
    ║  knowledge/{domain}/{knowledge_type}_{id}.md                  ║
    ╚═══════════════════════════════════════════════════════════════╝
╚═══════════════════════════════════════════════════════════════════════╝
```

Knowledge Capture Flow

```
Task Output
    │
    ▼
┌─────────────────────────────────────────┐
│         Knowledge Capture               │
│                                         │
│  1. Analyze output for patterns         │
│     └─ Keyword matching                 │
│     └─ Output structure detection       │
│                                         │
│  2. Classify knowledge type             │
│     └─ landing_page_pattern             │
│     └─ pricing_strategy                 │
│     └─ architecture_pattern             │
│     └─ marketing_strategy               │
│     └─ coding_pattern                   │
│     └─ sales_approach                   │
│     └─ design_pattern                   │
│     └─ workflow_process                 │
│                                         │
│  3. Extract insights                    │
│     └─ Code blocks                      │
│     └─ JSON structures                  │
│     └─ Structured content               │
│                                         │
│  4. Generate embedding                  │
│     └─ OpenAI (if API key)              │
│     └─ TF-IDF fallback                  │
│                                         │
│  5. Store in database                   │
│     └─ SQLite entry                     │
│     └─ Markdown file                    │
└─────────────────────────────────────────┘
```

Knowledge Retrieval Flow

```
Task Description
    │
    ▼
┌─────────────────────────────────────────┐
│         Knowledge Retrieval             │
│                                         │
│  1. Generate query embedding            │
│                                         │
│  2. Fetch embeddings from database      │
│     ├─ Filter by domain (optional)      │
│     ├─ Filter by type (optional)        │
│                                         │
│  3. Compute cosine similarities         │
│                                         │
│  4. Rank by similarity                  │
│     ├─ Filter by min_similarity (0.3)   │
│     ├─ Return top_k (default: 5)        │
│                                         │
│  5. Format context for injection        │
│     ├─ Markdown formatting              │
│     ├─ Truncation if needed             │
│     ├─ Source attribution               │
└─────────────────────────────────────────┘
    │
    ▼
Agent Context (enhanced with prior knowledge)
```

Reputation Scoring Algorithm

```python
def get_agent_score(agent_name: str) -> float:
    """
    Score components (total = 1.0):
    - Success rate: 60% weight
    - Experience: 20% weight (logarithmic scale)
    - Efficiency: 20% weight (faster = better)
    """
    rep = get_reputation(agent_name)

    if not rep:
        return 0.5  # Neutral for new agents

    total_tasks = rep.tasks_completed + rep.tasks_failed

    # Success component (60%)
    success_component = rep.success_rate * 0.6

    # Experience component (20%) - log scale
    experience_component = min(1.0, log(total_tasks + 1) / 5) * 0.2

    # Efficiency component (20%) - 60s baseline
    if rep.average_execution_time > 0:
        efficiency = min(1.0, 60 / rep.average_execution_time)
    else:
        efficiency = 0.5
    efficiency_component = efficiency * 0.2

    return success_component + experience_component + efficiency_component
```

Reputation-Enhanced Agent Selection

```
Task
    │
    ▼
┌──────────────────────────────────────────┐
│     Capability Index (Enhanced)          │
│                                          │
│  1. Get base capability matches          │
│     └── Division matching                │
│     └── Keyword overlap                  │
│     └── Technical skill matching         │
│                                          │
│  2. For each match, get reputation       │
│     └── Fetch from agent_reputation      │
│     └── Calculate score (0-1)            │
│                                          │
│  3. Combine scores                       │
│     └── capability_score * 0.7           │
│     └── reputation_score * 0.3           │
│                                          │
│  4. Sort by combined score               │
│                                          │
│  5. Return ranked agents                 │
└──────────────────────────────────────────┘
```

Module Structure

```
orchestration/
├── database.py           # SQLite manager for knowledge & reputation
│   ├── DatabaseManager   # Connection pooling, CRUD operations
│   ├── KnowledgeEntry    # Knowledge data class
│   └── AgentReputation   # Reputation data class
│
├── embeddings.py         # Text embedding system
│   ├── OpenAIEmbeddings  # OpenAI provider
│   ├── SimpleTFIDFEmbeddings  # Fallback provider
│   └── EmbeddingSystem   # Provider selection, similarity search
│
├── knowledge_capture.py  # Knowledge extraction
│   ├── KNOWLEDGE_PATTERNS  # Pattern definitions
│   ├── KnowledgeCapture  # Main capture class
│   └── store_agent_knowledge()  # Convenience function
│
├── knowledge_retrieval.py  # Knowledge search
│   ├── KnowledgeRetrieval  # Search and formatting
│   ├── retrieve_relevant_knowledge()  # Convenience function
│   └── get_knowledge_context()  # For task injection
│
├── reputation.py         # Performance tracking
│   ├── ReputationManager  # Scoring and ranking
│   ├── update_agent_reputation()  # After task completion
│   └── rank_agents_by_reputation()  # For selection
│
└── capability_index.py   # Enhanced with reputation
    ├── match_task_with_reputation()  # Combined matching
    └── select_best_agent()  # Single best selection
```

API Endpoints

```
/api/analytics/agent-performance (GET)
    │
    └── Returns:
        ├── totalAgents
        ├── totalTasksCompleted
        ├── totalTasksFailed
        ├── overallSuccessRate
        ├── averageExecutionTime
        ├── topPerformers[]
        │   ├── agent_name
        │   ├── success_rate
        │   └── tasks_completed
        ├── recently Active[]
        └── performanceByDivision{}
```

Data Persistence

```
.data/
└── mission_control.db     # SQLite database (WAL mode)
    ├── knowledge_entries  # Captured knowledge
    └── agent_reputation   # Performance metrics

knowledge/
├── README.md
├── marketing/
│   └── landing_page_pattern_*.md
├── product/
│   └── pricing_strategy_*.md
├── engineering/
│   └── architecture_pattern_*.md
├── sales/
├── design/
└── operations/
```

Configuration

```yaml
# config.yaml

knowledge:
  enabled: true
  min_confidence: 0.3
  max_context_entries: 5
  knowledge_dir: knowledge

embeddings:
  provider: auto          # auto, openai, tfidf
  openai_model: text-embedding-3-small
  tfidf_dimension: 256

reputation:
  enabled: true
  reputation_weight: 0.3  # vs capability
  min_tasks_threshold: 3  # before affecting ranking
```

Learning Workflow Example

```
Project 1: Build Landing Page
    │
    ├── Task executes successfully
    │
    ├── Knowledge Capture detects:
    │   └── landing_page_pattern (confidence: 0.7)
    │
    ├── Extracts insights:
    │   └── Hero section structure
    │   └── CTA placement patterns
    │
    ├── Generates embedding
    │
    └── Stores in knowledge_entries + knowledge/marketing/

    ...6 months later...

Project 2: Build Marketing Website
    │
    ├── Task planner receives task
    │
    ├── Knowledge Retrieval:
    │   └── Query: "build marketing website landing page"
    │   └── Finds: landing_page_pattern (similarity: 0.85)
    │
    ├── Injects into context:
    │   └── "Based on previous projects, successful
    │        landing pages use: [hero section pattern]..."
    │
    └── Agent executes with enhanced context
```